

Local Docker Stack

n8n · Ollama · Qdrant

A step-by-step guide to running a local AI automation stack using Docker Compose. Everything runs on your own machine — no cloud, no external AI services.

Works on: macOS, Windows (with WSL2), Linux

1. What is Docker?

Docker is a tool that lets you run software in isolated containers — think of each container as a lightweight, self-contained computer that runs one specific service. Instead of installing n8n, Ollama and Qdrant directly on your machine (which can cause version conflicts and messy setups), Docker keeps each service in its own box.

Docker Compose is a tool that lets you define and run multiple containers together using a single configuration file (docker-compose.yml). You describe all your services in that file and start them all with one command.

2. Install Docker

macOS

Download and install Docker Desktop from:

```
https://www.docker.com/products/docker-desktop/
```

Open the installer, drag Docker to Applications, and launch it. Docker Desktop includes both Docker and Docker Compose. Wait until the whale icon in the menu bar stops animating — Docker is ready.

Windows

Install WSL2 first (required for Docker on Windows):

```
# Run in PowerShell as Administrator:  
wsl --install  
# Restart your computer, then install Docker Desktop from:  
# https://www.docker.com/products/docker-desktop/
```

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) \
    signed-by=/etc/apt/keyrings/docker.gpg] \
    https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
    sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
    docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
# Log out and back in, then verify:
docker --version && docker compose version
```

Verify Docker is working:

```
docker --version
# Expected: Docker version 29.x.x

docker compose version
# Expected: Docker Compose version v2.x.x
```

3. Create the Project Folder

Create a dedicated folder for the stack. All data will be stored here.

```
# macOS / Linux:
mkdir -p ~/my-stack
cd ~/my-stack

# Windows (in WSL2 terminal):
mkdir ~/my-stack
cd ~/my-stack
```

After setup, your folder will look like this:

```
my-stack/
■■■ docker-compose.yml    ← the main config file you create
■■■ n8n_data/             ← n8n workflows and credentials (auto-created)
■■■ ollama_data/          ← AI models (auto-created, can be large)
■■■ qdrant_data/          ← vector database storage (auto-created)
```

4. Create the docker-compose.yml

This is the main configuration file. It tells Docker which containers to run, how they connect to each other, and where to store their data.

Create the file:

```
# macOS / Linux:
nano docker-compose.yml

# Or open the folder in VS Code and create the file there
```

Paste the following content exactly:

```
services:

  n8n:
    image: n8nio/n8n:latest
    container_name: n8n
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=0.0.0.0
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      - GENERIC_TIMEZONE=Europe/Amsterdam # change to your timezone
      - TZ=Europe/Amsterdam
      - N8N_SECURE_COOKIE=false
    volumes:
      - ./n8n_data:/home/node/.n8n
    networks:
      - stack_net
```

```
ollama:
  image: ollama/ollama:latest
  container_name: ollama
  restart: unless-stopped
  ports:
    - "11434:11434"
  volumes:
    - ./ollama_data:/root/.ollama
  networks:
    - stack_net

qdrant:
  image: qdrant/qdrant:latest
  container_name: qdrant
  restart: unless-stopped
  ports:
    - "6333:6333"
  volumes:
    - ./qdrant_data:/qdrant/storage
  networks:
    - stack_net

networks:
  stack_net:
    driver: bridge
```

N8N_SECURE_COOKIE=false is set because we are running locally over HTTP. On a production server with HTTPS this should be set to true.

5. Start the Stack

From inside the my-stack folder, run:

```
docker compose up -d

# The -d flag runs everything in the background (detached mode)
# First run will download the Docker images – this may take a few minutes
```

Check that all containers are running:

```
docker compose ps

# You should see:
# NAME          STATUS
# n8n            Up
# ollama        Up
# qdrant        Up
```

Open n8n in your browser:

```
http://localhost:5678
```

Create your n8n owner account on the first visit (email + password).

6. Download AI Models

Once Ollama is running, pull the models you need. These are downloaded once and stored in the ollama_data folder.

Classification model — Gemma 4 E4B QAT (~6.1 GB):

```
docker exec -it ollama ollama pull gemma4:e4b-it-qat
```

Draft generation model — Llama 3.1 8B (~4.7 GB):

```
docker exec -it ollama ollama pull llama3.1:8b
```

Embedding model for RAG — nomic-embed-text (~274 MB):

```
docker exec -it ollama ollama pull nomic-embed-text
```

Verify all models downloaded correctly:

```
docker exec -it ollama ollama list
```

```
# Expected output:
# NAME                SIZE
# gemma4:e4b-it-qat   6.1 GB
# llama3.1:8b         4.7 GB
# nomic-embed-text    274 MB
```

7. Connect n8n to Ollama and Qdrant

Inside n8n, when adding credentials for Ollama or Qdrant, do not use localhost. Instead, use the container name. All three containers are on the same Docker network (stack_net), so they can find each other by name:

Service	URL to use in n8n
Ollama	<code>http://ollama:11434</code>
Qdrant	<code>http://qdrant:6333</code>

Why not localhost? Each Docker container has its own network namespace. From inside the n8n container, localhost refers to n8n itself — not the host machine. Container names are resolved automatically within the shared network.

8. Useful Commands

Everyday management:

```
# Start the stack
docker compose up -d

# Stop the stack
docker compose down

# Restart one container
docker compose restart n8n

# View logs
docker compose logs -f n8n

# Check status and memory usage
docker compose ps
docker stats --no-stream
```

Ollama models:

```
# List installed models
docker exec -it ollama ollama list

# Download a model
docker exec -it ollama ollama pull MODEL_NAME

# Remove a model
docker exec -it ollama ollama rm MODEL_NAME

# Test a model
docker exec -it ollama ollama run llama3.1:8b
# Type /bye to exit
```

Update all images to the latest version:

```
docker compose pull
docker compose up -d
```

9. Optional: Adding Caddy for HTTPS

By default the stack runs locally over HTTP on localhost — which is perfectly fine for development and testing on your own machine. Caddy is only needed if you want to make n8n accessible over the internet

with a secure HTTPS connection, for example when deploying to a server that multiple people need to reach.

When do you need Caddy?

You do NOT need Caddy if:

- You are running the stack on your own laptop for local development
- Only you need access to n8n (via localhost:5678)
- You are testing workflows before deploying to a server

You DO need Caddy if:

- You are deploying to a VPS or server
- Multiple people need to access n8n from different locations
- You want a proper domain name with a green padlock (HTTPS)
- You need webhooks to work from external services (Gmail, Salesforce etc.)

What does Caddy do?

Caddy sits in front of n8n and handles two things automatically: it routes incoming web traffic from your domain to the n8n container, and it requests and renews a valid HTTPS certificate from Let's Encrypt — all without any manual configuration. You just give it a domain name and it takes care of the rest.

Prerequisites for Caddy:

- A domain name with an A-record pointing to your server IP
- Ports 80 and 443 open in your firewall
- The stack running on a publicly accessible server (not localhost)

Step 1 — Create the Caddyfile:

```
nano ~/my-stack/Caddyfile

# Paste this, replacing with your actual domain:
your-domain.com {
    reverse_proxy n8n:5678
}
```

Step 2 — Add Caddy to docker-compose.yml:

Add the following service block to your docker-compose.yml, inside the services: section:

```
caddy:
  image: caddy:latest
  container_name: caddy
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./Caddyfile:/etc/caddy/Caddyfile
    - ./caddy_data:/data
    - ./caddy_config:/config
  networks:
    - stack_net
```

Step 3 — Remove the direct port mapping from n8n:

Once Caddy is in place, all traffic goes through Caddy — not directly to n8n. Remove the ports section from the n8n service in docker-compose.yml:

```
# Remove these two lines from the n8n service:
ports:
  - "5678:5678"
```

Step 4 — Update n8n environment variables:

Update the n8n environment section to use your domain and HTTPS:

```
environment:
  - N8N_HOST=your-domain.com
  - N8N_PORT=5678
  - N8N_PROTOCOL=https
  - N8N_SECURE_COOKIE=true
  - WEBHOOK_URL=https://your-domain.com/
  - GENERIC_TIMEZONE=Europe/Amsterdam
  - TZ=Europe/Amsterdam
```

Step 5 — Restart the stack:

```
docker compose up -d

# Wait 30-60 seconds for Caddy to request the HTTPS certificate
# Then open your browser at: https://your-domain.com
```

Caddy automatically renews the certificate before it expires — you never need to manage SSL certificates manually.

10. Troubleshooting

n8n shows a secure cookie error

Set `N8N_SECURE_COOKIE=false` in `docker-compose.yml` under the `n8n` environment section. This is normal when running locally over HTTP (not HTTPS).

Can't connect to Ollama from n8n

Make sure you are using `http://ollama:11434` as the base URL in the Ollama credential — not `http://localhost:11434`. Containers communicate via their names, not localhost.

The computer becomes slow or unresponsive when running a model

The model is using too much RAM. Run `docker stats` to check memory usage. Consider using a smaller model (e.g. `llama3.2:3b` instead of `llama3.1:8b`), or close other applications while running inference.

Containers stop after restarting the computer

Run `docker compose up -d` again from the `my-stack` folder. Alternatively, make sure `restart: unless-stopped` is set for each service in `docker-compose.yml` — this restarts containers automatically when Docker starts.

Port already in use error

Another application is using port 5678, 11434, or 6333. Either stop the other application, or change the left-hand port number in `docker-compose.yml` (e.g. change `5678:5678` to `5679:5678` and access n8n at `localhost:5679`).